BENCHMARK REPORT

Altair vs Express · Fiber · Ruby

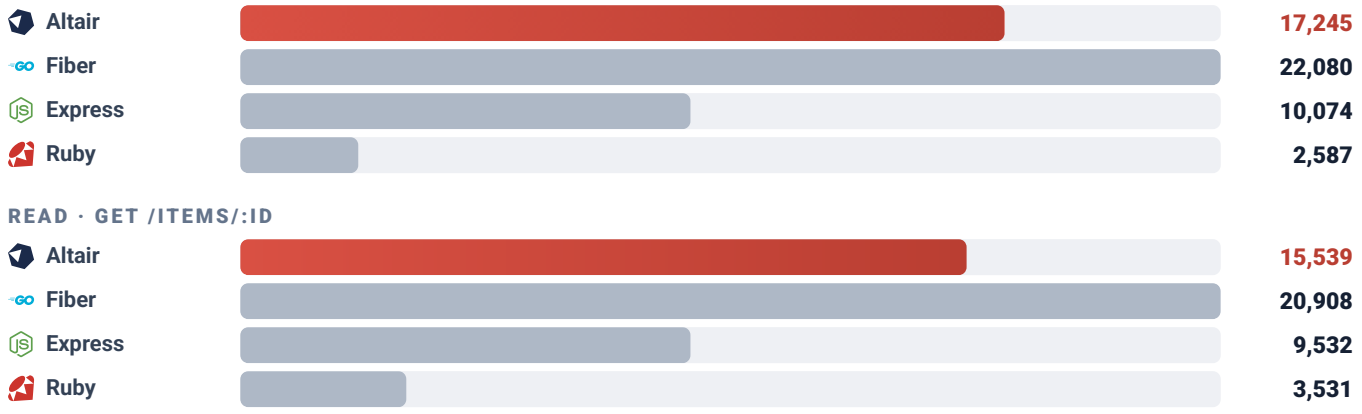
v0.4.3 · Crystal 1.22.1
October 24, 2026
arab-open-source.github.io/Altair

VIRTUAL USERS	DURATION	DB BUDGET	WORKERS	DATABASE	DRIVER
100	60 s	20 conns	6	PostgreSQL	k6

HEAD-TO-HEAD

FRAMEWORK	WRITE REQ/S	WRITE P99	WRITE MAX	READ REQ/S	READ P99	READ MAX
Altair Crystal	17,245	8.6 ms	**18.1 ms** ★	15,539	14.6 ms	247.0 ms
Fiber Go	22,080	8.5 ms	24.8 ms	20,908	10.4 ms	25.1 ms
Express Node.js	10,074	16.2 ms	53.6 ms	9,532	20.5 ms	64.4 ms
Ruby full-stack Puma + ORM	2,587	337.7 ms	1,060 ms	3,531	257.5 ms	1,042 ms

THROUGHPUT



HIGHLIGHTS

18.1 ms
Best write max of all four arms — and the tightest p99.9 (10.7 ms), below even Go.

6.7×
Faster writes than the Ruby full-stack arm — the closest architectural peer (4.4× on reads).

1.7×
Faster than Express on writes, 1.6× on reads — with a 3x tighter write p99.

- **Fiber (Go) leads raw throughput**; Altair holds second place with a statistically tied write p99 (8.6 vs 8.5 ms).
- **Zero meaningful failures** across all eight runs (17 warmup-boundary resets on Express, 2 on Ruby).
- Altair's read tail carries one Boehm stop-the-world GC pause — the tracked next optimization target.

REPRODUCE

```
$ cd examples/benchmark_k6 && scripts/bench.sh altair | fiber | express | ruby
```

Defaults: 100 VUs, 60 s per phase, discarded warm-up, 20-connection / 6-worker budget for every framework. Raw k6 exports land in results/.

Harness: examples/benchmark_k6 — k6 summary exports committed under results/

Methodology & full report: examples/benchmark_k6/README.md